ENGINEX WORKBENCH

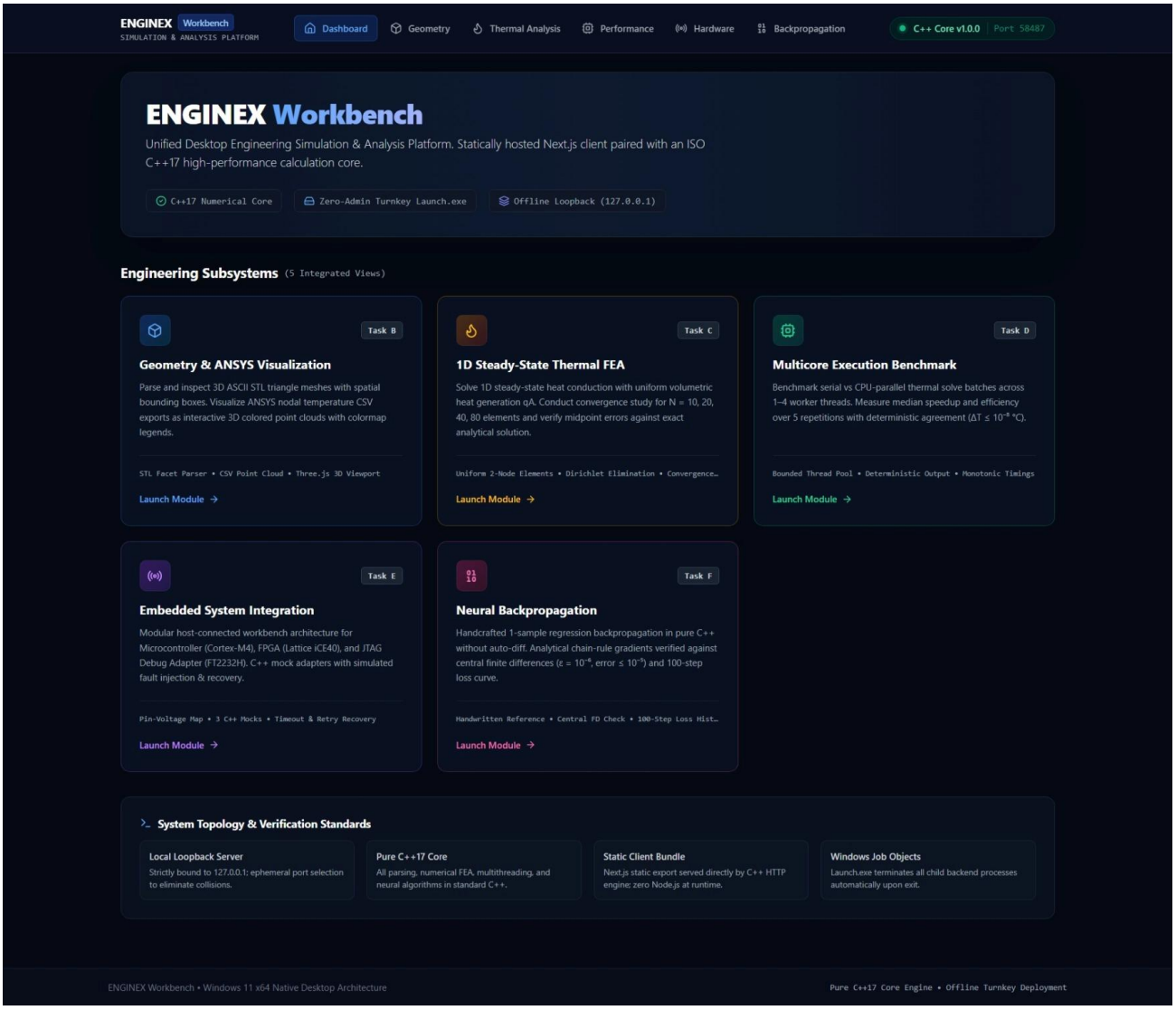
Screenshot Walkthrough

A screenshot-based alternative to the 3–5 minute demonstration video

00:00 — Dashboard

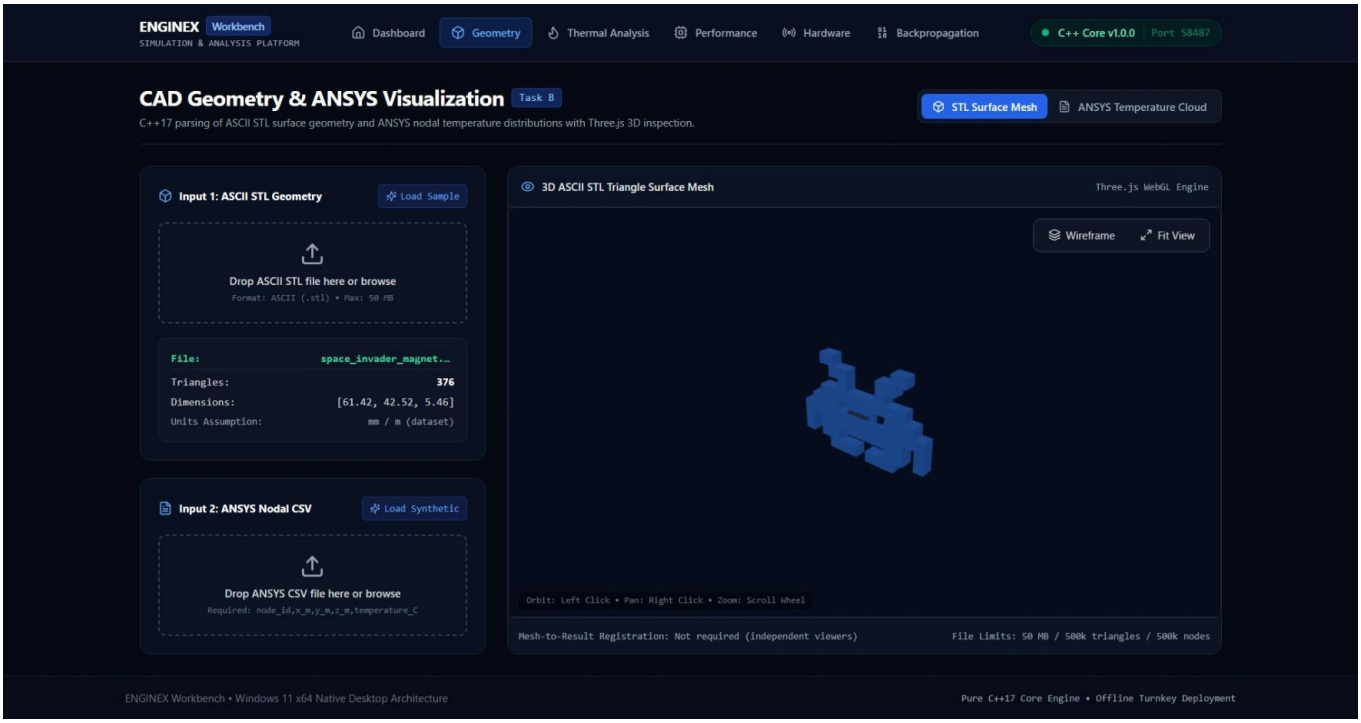
The ENGINEX Workbench dashboard provides access to the application's five engineering modules: Geometry, Thermal Analysis, Performance, Hardware and Backpropagation.

The dashboard acts as the central navigation interface, allowing users to access the required engineering operations from a single application.



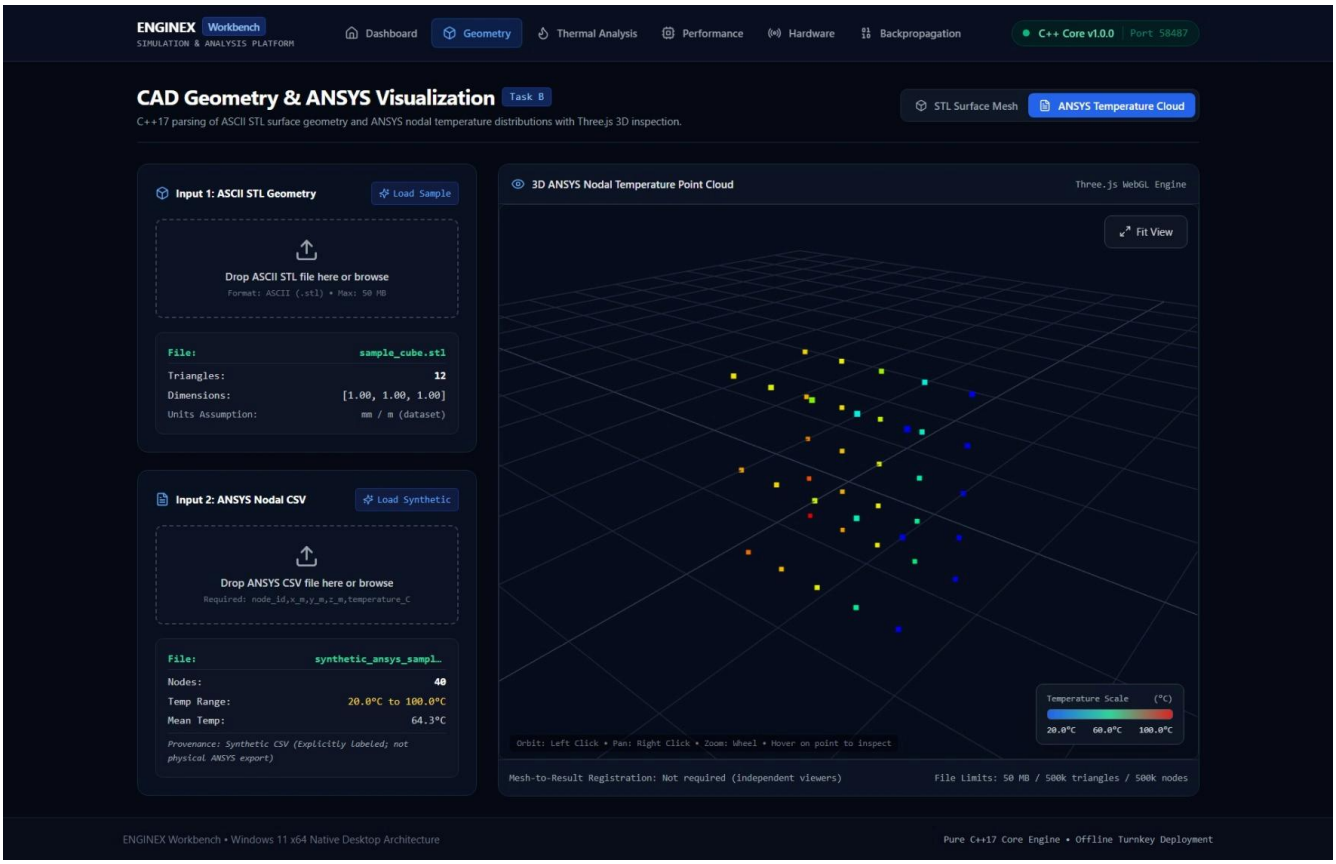
00:20 — ASCII STL geometry

The Geometry module provides functionality for loading and visualising ASCII STL files. STL files represent three-dimensional surfaces using triangular facets. The C++ backend is responsible for parsing the input file and extracting the facet and vertex information. The parsed geometry is validated before being made available to the frontend. The frontend then displays the resulting three-dimensional model in the geometry viewer. This allows users to inspect the structure represented by the input file. The supplied sample STL model is used to test the geometry-loading workflow. The implementation focuses on ASCII STL parsing rather than claiming support for every STL variant.



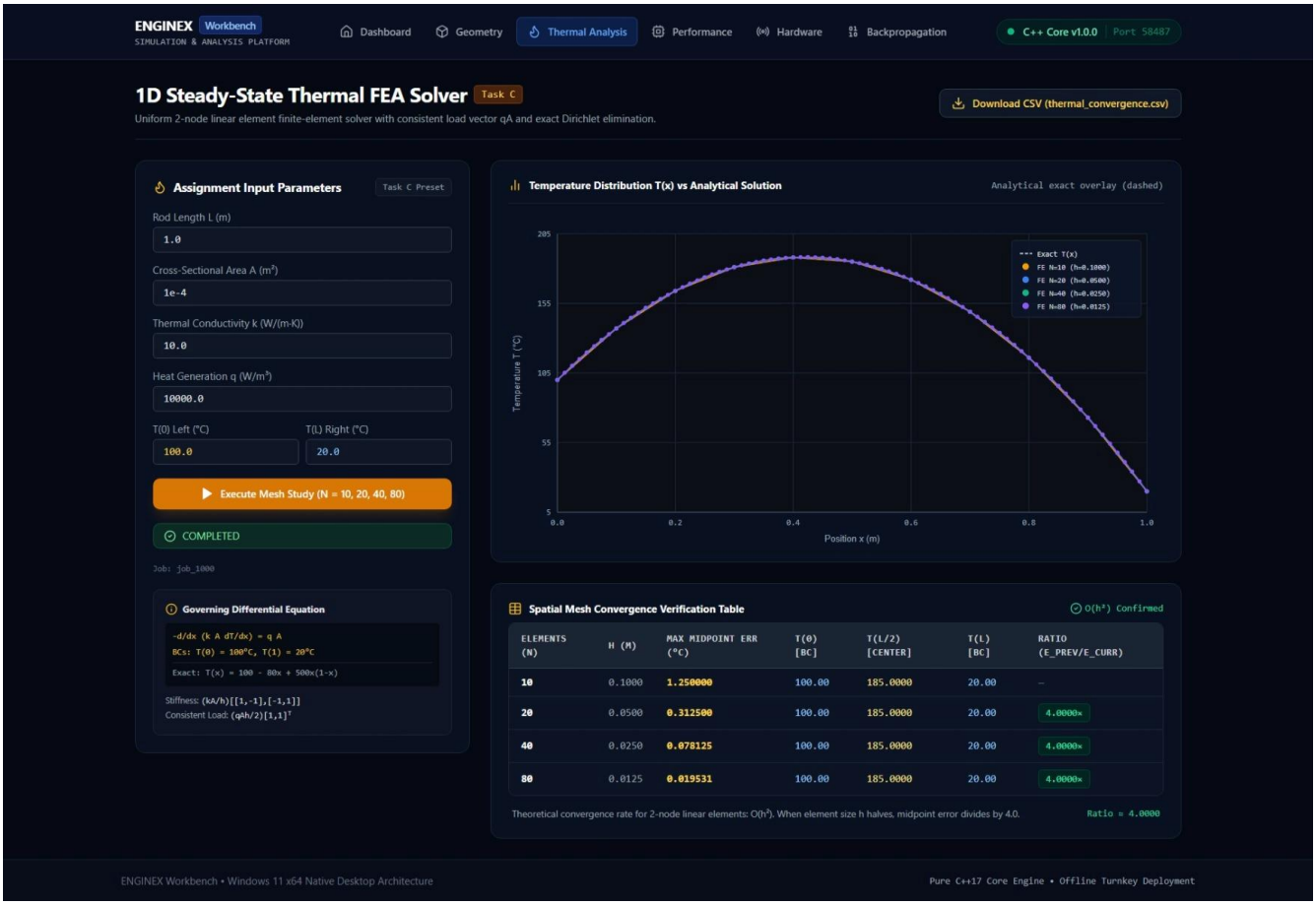
00:45 — Nodal temperature point cloud

The Geometry module also supports CSV-based nodal temperature data. The input file contains coordinate information and corresponding temperature values for individual nodes. The C++ backend processes the CSV content and validates the required numerical fields. After processing, the data is transferred to the frontend for visualization. The point-cloud representation displays the spatial distribution of temperature values across the supplied nodes. This provides a way to inspect engineering data independently of the STL geometry viewer. The included dataset is a synthetic sample fixture used for demonstrating the visualization functionality.



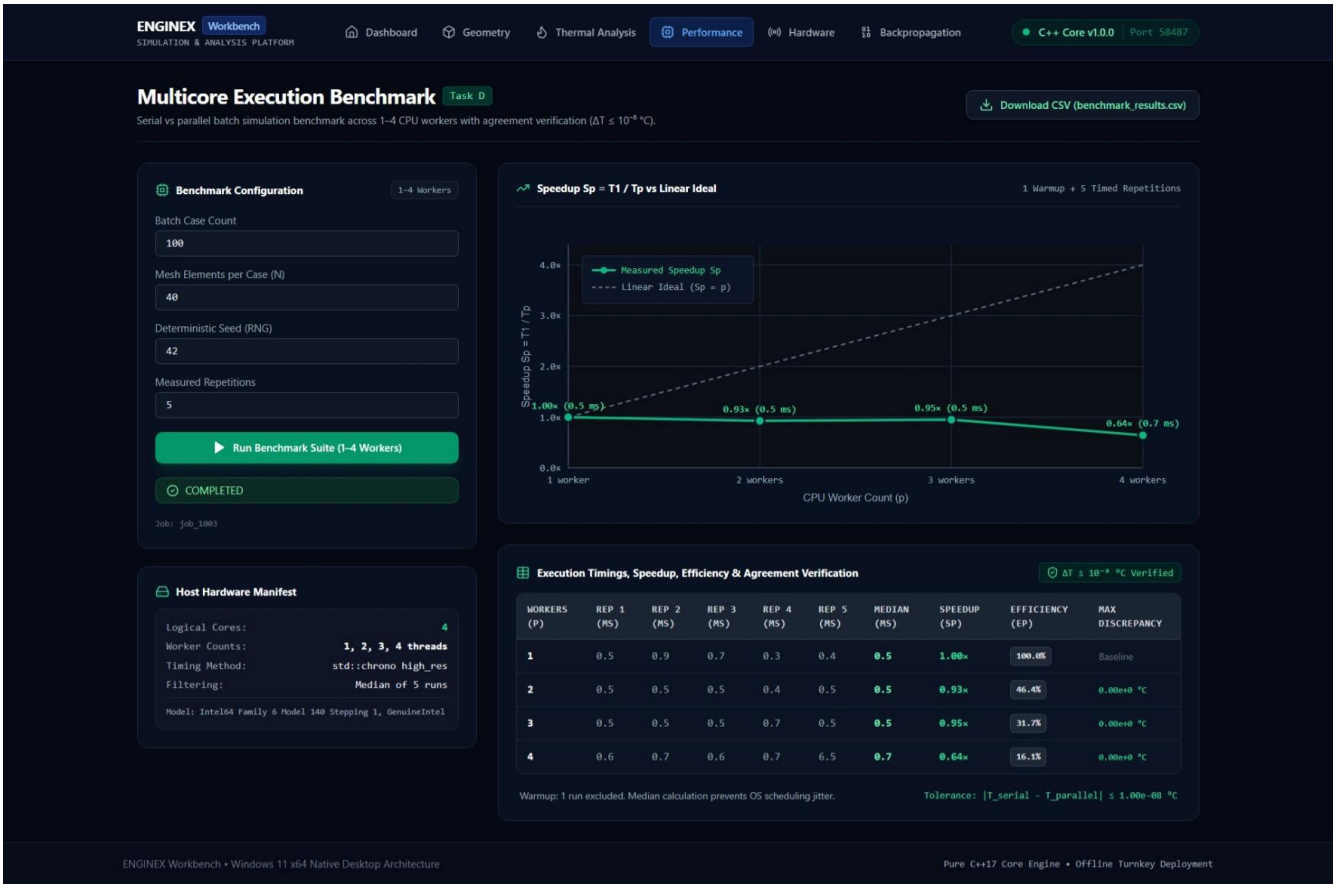
01:15 — Thermal FEA and mesh convergence

The Thermal Analysis module implements steady-state one-dimensional heat conduction using the Finite Element Method. The model represents heat transfer through a uniform rod with prescribed boundary temperatures and uniform volumetric heat generation. The solver uses two-node linear elements to discretize the domain. Element stiffness matrices and source vectors are assembled into the global system. Dirichlet boundary conditions are applied before solving for the nodal temperatures. The application supports mesh studies using 10, 20, 40 and 80 elements. The computed results are compared against the analytical temperature distribution. Maximum midpoint interpolation error is also evaluated to examine the effect of mesh refinement.



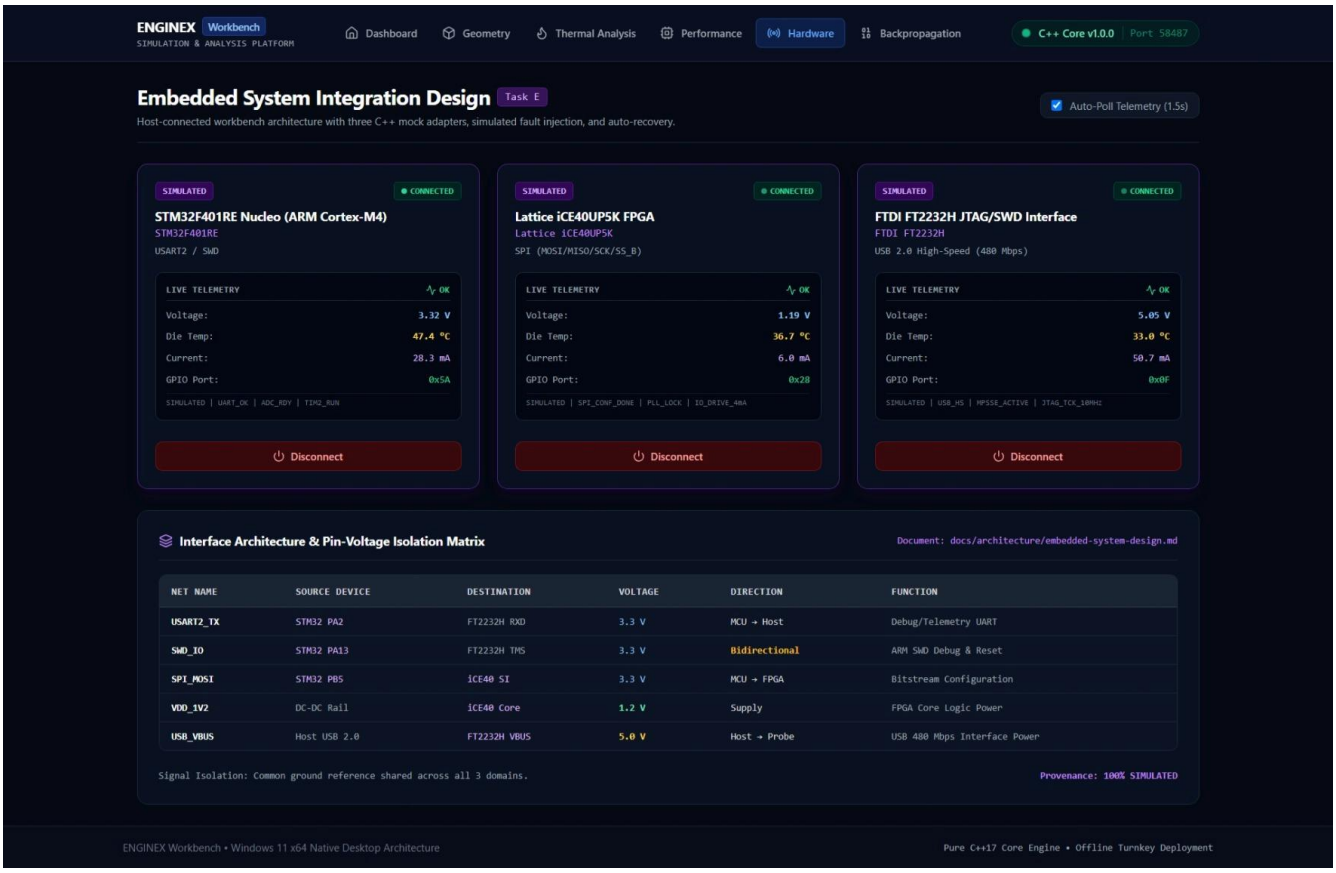
02:00 — Serial/parallel performance benchmark

The Performance module evaluates computational execution using serial and CPU-parallel processing. It uses repeated computational workloads to compare execution across different worker counts. A fixed seed is used to generate reproducible benchmark inputs. The implementation includes a bounded worker pool to control parallel task execution. A warm-up run is performed before the measured repetitions. Median execution times are used to reduce the influence of individual timing variations. Speedup and parallel efficiency are calculated to evaluate the effectiveness of multicore execution. The results also illustrate that parallel processing may introduce overhead for smaller workloads. Performance therefore depends on workload size and the available CPU resources.



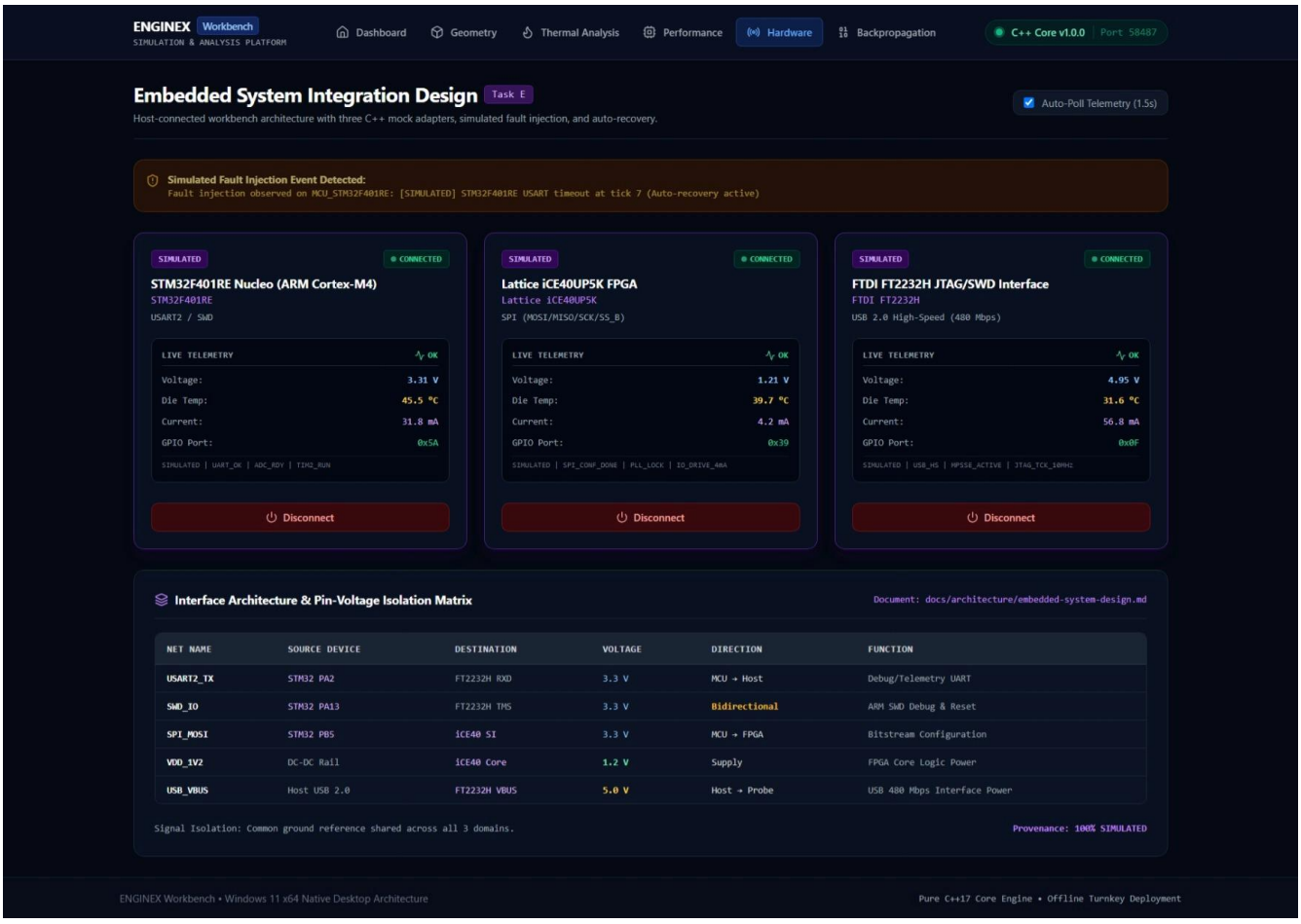
02:45 — Embedded hardware simulation

The Hardware module provides a simulated device-management interface for three embedded-system components: STM32F401RE Nucleo, Lattice iCE40UP5K FPGA and FTDI FT2232H USB debug adapter. Each device is represented through a mock adapter within the C++ backend. The module provides device discovery and connection-management functionality. Users can view the connection state of the available simulated devices. Telemetry information is also displayed through the application interface. The adapter structure separates device-related operations from the frontend presentation. This provides a consistent software interface for managing the simulated devices. No physical hardware communication or board-level testing is claimed.



03:20 — Simulated fault injection and recovery

The Hardware module includes a simulated fault-injection mechanism for representing communication failures. A timeout can be triggered within the mock device adapter to reproduce an error condition. The application displays an error notification and updates the corresponding device status. The mock adapter provides a recovery workflow to handle the simulated failure. Subsequent telemetry updates can indicate that the simulated connection has resumed. This functionality helps exercise the application's error-reporting and device-state handling. The fault-injection mechanism is implemented at the software level. It does not represent physical electrical faults or validate recovery on an actual embedded device.



04:00 — Backpropagation and gradient verification

The Backpropagation module implements a single-hidden-unit neural network using the tanh activation function. The model uses one input, one hidden neuron and one output neuron. The forward pass calculates the hidden activation, predicted output and loss value. During the backward pass, analytical gradients are calculated for all four parameters using the chain rule. Gradient descent is then applied using the specified learning rate. All parameter updates use gradients calculated from the same initial parameter values. Central finite differences are used to compare numerical gradients with the analytical derivatives. The application also displays the reduction in training loss over successive iterations. The handwritten mathematical derivation is provided separately as a two-page appendix.

